

OneCheck 발표 준비

가면서 읽기

2026.02.11 발표용 | PPT 12장 + 기술 상세 + 예상 Q&A 전부 포함

PART 1. 30 초 요약

"고객사 엑셀 명부를 올리면, AI가 자동으로 검증하고, 틀린 부분을 웹에서 바로 고치고, 텔레그램으로 결과를 받는 플랫폼입니다."

핵심 키워드 3개: **편리성 / 자동화 / 정확성**

- **편리성:** 메일 왕복 없이 웹에서 5 분
- **자동화:** 컬럼 매핑 → 3 단계 검증 → 리포트 → 알림까지 전부 자동
- **정확성:** 기존 매크로 21 개 100% + AI 맥락적 판단 + 쓸수록 학습

PART 2. 슬라이드별 발표 가이드 (10 분)

슬라이드 1: 표지 (15 초)

"OneCheck — Zero Touch. 퇴직연금 검증 자동화 플랫폼입니다."

편리성, 자동화, 정확성 세 키워드만 읽고 넘기기.

슬라이드 2: 문제 (1 분)

"지금 우리 프로세스는 파일 하나에 3~4시간, 왕복하면 1~2일입니다."

고객사 → 메일로 엑셀 전송 → 우리가 매크로 돌림 (2~3시간)

→ 오류 있으면 "수정해주세요" 메일 → 고객사 수정 → 다시 매크로...

→ 2~3회 반복

꼭 말할 것:

- **느리다:** 파일 하나에 2~3 시간, 왕복까지 1~2 일
- **위험하다:** 개인정보 든 엑셀이 메일로 왔다갔다
- **사람에 의존:** 검증 기준이 담당자마다 다름
- **컬럼명 문제:** 고객사마다 "급여/기준급여/월급여/연봉" 전부 다름

슬라이드 3: 해결 (1 분)

"파일 올리면 나머지는 전부 자동입니다."

기존	OneCheck
3~4시간	5분
메일 왕복	웹에서 바로
매크로 수동	AI + 규칙 자동
학습 불가	쓸수록 향상

슬라이드 4: UI — 로그인 + 진단 질문 (45 초)

"로그인하면 바로 검증 모드를 고르고, 7페이지 위자드로 진단 질문에 답합니다."

꼭 말할 것:

- 2 가지 모드: Basic(빠른 확인) / Complete(정밀 검증)
- 7 페이지 74 개 질문: 단계별로 깔끔하게
- 자동 합계: 임원+직원+계약직 → 합계 자동
- 이 답변이 검증 기준이 됨: "정년 60 세" 입력 → 60 세 넘는 직원 자동 탐지

진단 질문 7섹션:

섹션	내용	질문 수
1	일반사항 (회사명, 담당자)	6
2	평가대상/퇴직급여 (인원수, 금액)	26
3	사외적립자산	7
4	제도 주요사항 (정년, 급여체계)	11
5	할인율	11
6	적립비율	8
7	특이사항	5

슬라이드 5: UI — AI 매핑 + 3 색 결과 (45 초)

"파일 올리면 AI가 고객 컬럼을 표준 컬럼에 자동 매핑합니다."

AI 매핑 예시:

- "사번" → "사원번호" (95%)
- "기준급여(원)" → "기준급여" (92%)

- 매핑 안 되면 수동 드롭다운 선택

3색 결과:

● 93 오류 ● 13 경고 ● 644 정상 신뢰도 87.5%

슬라이드 6: UI — 값 수정/값 유지 + 스프레드시트 (45 초)

"오류마다 두 가지 선택. 수정할 거면 '값 수정', 오류가 아니면 '값 유지'."

핵심:

- 값 수정 → 스프레드시트 편집기 열림, 엑셀처럼 편집
- 값 유지 → "이건 오류 아님" → AI 가 학습, 다음에 안 잡음
- 스프레드시트: 오류 셀 빨간 표시, Undo/Redo, "적용" → 자동 재검증

슬라이드 7: UI — 대시보드 + 관리자 + 알림 (45 초)

"대시보드에서 전체 현황 한눈에, 텔레그램으로 즉시 알림."

기능	설명
대시보드	총 건수, 성공/실패 비율, 최근 이력
관리자	사용자/학습데이터/설정 관리
텔레그램	검증 완료 즉시 알림 + 승인/거부 버튼

슬라이드 8: 로직 — 3 단계 검증 (1 분 30 초) ★핵심★

"확실한 건 코드로, 복잡한 건 AI로. 3단계입니다."

- L1 형식 검증 (0.1초) → 하드코딩 21개 룰 (기존 매크로 100% 커버)
 L2 교차 검증 (0.5초) → 진단 답변 vs 명부 자동 비교

L3 AI 검증 (3~5초) → 정년초과, 패턴 이상, 맥락 추론

L1 — 21 개 룰 (기존 매크로 완전 대체)

#	룰
1-3	금액 음수 체크 (당년도/차년도 퇴직금, 중간정산)
4	입사연령 17~70세
5	입사일 > 생년월일
6-8	날짜 순서 (중간정산일 > 입사일, 시산일 관계)
9-11	날짜 형식 오류
12	기준급여 >= 최저임금 (2,060,740원)
13-17	필수값 누락 (사원번호/생년월일/입사일/급여/직종)
18-20	임원/계약직 퇴직금 필수 체크
21	재직자 중복 체크

L2 — 교차검증

- 인원수: 사용자 답변 vs 명부 자동 집계 (정확 일치)
- 금액: 사용자 답변 vs 명부 합계 ($\pm 5\%$ 허용)

L3 — AI

- 정년초과 탐지, 임원-급여 이상, 미지의 패턴 추론

반드시 말할 것:

"기존 매크로 21개 에러체크를 L1에서 100% 커버합니다. 거기에 L2 교차검증과 L3 AI를 더 했습니다."

슬라이드 9: 로직 — AI + 자율 학습 (1 분)

"매크로로는 불가능한 것 + 쓸수록 정확해지는 것."

매크로	OneCheck
규칙만	맥락적 추론
학습 불가	클릭 한 번으로 학습
고정 21개	미지의 오류도 탐지

학습 흐름:

"값 유지" 클릭 → DB 저장 → 다음 검증 시 자동 억제

실제 결과:

학습 전: 오류 97건 → 학습 후: 오류 4건 (93건 자동 억제)

"개인정보(주민번호/이름/전화번호) 없이 이 모든 검증을 수행합니다."

슬라이드 10: 차별화 + 보안 (1 분)

"왜 우리가 다른가."

	일반	OneCheck
검증	AI만 or 규칙만	하이브리드
매크로 호환	새로 만듦	21개 100%
AI 없을 때	작동 안 함	100% 동작
개인정보	필요	PII 불필요

보안:

- 검증에 주민번호/이름/전화번호 불필요 → 수집 자체를 안 함
- 만약 포함되면 → PII 자동 탐지 & 마스킹
- Graceful Degradation:** OpenAI 없어도 규칙 기반 100% 동작

슬라이드 11: 비전 (30 초)

"OneCheck는 시작점입니다."

명부 검증 (완성) → 재정 검증 (예정) → 퇴직급여 계산 (목표)

파일 하나 → 검증 → 계산 → 리포트까지 **원패스**.

슬라이드 12: 감사합니다 + Q&A

PART 3. 예상 질문 & 답변 (★중요★)

Q1. "기존 매크로랑 뭐가 달라요?"

"기존 매크로 21개 에러체크를 L1에서 100% 구현했습니다. 거기에 교차검증(L2)과 AI(L3)를 더한 거예요. 매크로는 형식만 보는데, 우리는 '인원수가 답변과 맞는지', 'AI가 볼 때 이상한 패턴은 없는지'까지 봅니다."

Q2. "AI 없으면 어떻게 돼요?"

"Graceful Degradation 설계입니다. OpenAI API 없어도 L1(규칙 21개) + L2(교차검증)는 100% 동작합니다. AI 매칭도 규칙 기반 fallback이 있어서 작동은 합니다."

Q3. "개인정보 문제는?"

"설계 단계부터 PII가 필요 없게 만들었습니다. 검증에 필요한 건 사원번호, 생년월일, 입사일, 급여, 퇴직금인데 여기에 주민번호나 이름은 불필요합니다. 만약 포함돼도 PII 자동 탐지 & 마스킹 모듈이 있습니다."

Q4. "학습이 뭔데요?"

"사용자가 '이건 오류 아님'을 클릭하면 패턴이 DB에 저장됩니다. 다음에 같은 패턴이 나오면 자동 억제돼요. 세라젼 파일에서 97건 오류 중 93건에 '값 유지' 누르면, 다음엔 4건만 뜹니다."

Q5. "3 단계 검증이 왜 필요해요?"

"확실한 건 빠르게, 복잡한 건 깊이 있게 보려고요."

- L1 (0.1 초): 날짜 형식, 음수, 필수값 — 확실한 오류를 빠르게
- L2 (0.5 초): 사용자 답변과 명부 비교
- L3 (3~5 초): AI가 전체 맥락 보고 판단

L1에서 잡힌 건 AI에 안 보내서 비용도 절감됩니다."

Q6. "컬럼 매핑은 어떻게?"

"3단계입니다.

- 규칙 기반: "사번" = "사원번호" 같은 alias 매칭
- AI 매칭: 규칙으로 안 되면 GPT-4o 에 요청
- 수동 매핑: 그래도 안 되면 드롭다운으로 선택

규칙으로 되는 건 규칙으로, 안 되는 것만 AI에 보내서 비용 최적화했습니다."

Q7. "5 분이 진짜예요?"

"시스템 자체는 5~10초입니다. 5분은 사용자가 진단 질문 답변하고 파일 올리는 시간까지 포함한 거예요. 기존 2~3시간 대비 5분입니다."

Q8. "다른 팀이랑 차이?"

4가지:

- 하이브리드: AI+규칙 둘 다 → 비용 효율 + 정확도
- 매크로 100% 호환: 업무 전환 비용 0
- 학습: 쓸수록 정확
- PII 불필요 설계: 개인정보 없이 검증 가능

Q9. "기술 스택?"

구분	기술
Frontend	React 18 + TypeScript + Vite
Backend	FastAPI (Python 3.11)
AI	OpenAI GPT-4o
DB	SQLite
알림	Telegram Bot API

구분	기술
보안	JWT, AES-256-GCM, RBAC

Q10. "앞으로 뭘 할 건가요?"

"명부 검증(완성) → 재정 검증(적립금/부채) → 퇴직급여 자동 산출. 최종 목표는 파일 하나로 원패스입니다."

Q11. "모드가 2 개인데 차이가 뭐예요?"

- Basic (roster): 명부 형식만 검증. 금액 관련 체크 스킵. 빠르게 확인용.
- Complete (full): 전체 21 개 룰 + 금액 비교 + 최저임금 체크. 정밀 검증용.

"Basic은 명부 구조만 빠르게 확인할 때, Complete는 제출 전 최종 검증할 때 씁니다."

Q12. "종업원구분이 뭔가요?"

"명부에서 사람을 분류하는 코드입니다."

- 1, 2 = 직원
- 3 = 임원
- 4 = 계약직

"임원/계약직은 당년도+차년도 퇴직금추계액이 필수입니다. 빈칸이면 오류로 잡아요."

PART 4. 기술 상세 (깊게 물어올 때)

4-1. 필수 필드 8 개

필드	타입	용도
사원번호	string	중복 체크, 고유 식별
생년월일	date	연령/정년 계산
성별	category	1/2 또는 M/F
입사일자	date	근속 계산
기준급여	number	최저임금 체크
당년도퇴직금추계액	number	핵심 검증 대상
종업원구분	category	임원/직원/계약직
근속년수	number	퇴직금 산출 기준

각 필드는 alias 매핑 지원 (사원번호 ← 사번/직원번호/employee_id)

4-2. 직원 인원 계산 방식

직원 = 전체 행 수 - 임원(코드3) - 계약직(코드4)

종업원구분이 빈칸이거나 비표준 코드여도 직원으로 간주.

→ 사용자 입력값과 명부 집계가 일치하게 됨.

4-3. 데이터 흐름 전체

```
[사용자]
├─ 진단 질문 답변 (74개)
├─ 엑셀 파일 업로드
│   ↓
│   [Excel Parser] → 헤더 자동 탐지 (한글 키워드 기반)
│   ↓
│   [Header Matcher] → 규칙 매칭 → (안 되면) AI 매칭
│   ↓
```

```

| [DataFrame 생성] → 표준 컬럼명 매핑
|   ↓
|   ┌ [L1 형식 검증] → 21개 룰
|   ├── [L2 교차 검증] → 답변 vs 집계
|   └── [L3 AI 검증] → GPT-4o
|       ↓
|   [중복 제거] → 같은 사원+같은 필드 = 하나로
|       ↓
|   [학습 패턴 적용] → false positive 억제
|       ↓
|   결과: { errors, warnings, checks, confidence }
|
| └ "값 수정" → SheetEditor → 재검증
| └ "값 유지" → 학습 DB 저장
| └ 다운로드 (검증 리포트 / 최종 명부)
|       ↓
|   [텔레그램 알림] → 승인/거부

```

4-4. 시스템 아키텍처

```

Frontend (React + TypeScript + Vite)
| REST API (axios)
|   ↓
Backend (FastAPI + Python 3.11)
├ core/validators/ → 3계층 검증
├ core/parsers/ → 엑셀 파싱
├ core/ai/ → AI 매칭, LLM
├ core/privacy/ → PII 탐지/마스킹
├ core/generators/ → 리포트 생성
├ core/database.py → SQLite (6개 테이블)
└ integrations/ → Telegram, n8n

```

DB 테이블:

- validation_history, cases, learned_patterns
- error_rules, behavior_events, system_settings

4-5. 학습 파이프라인 상세

```

"값 유지" 클릭
→ POST /api/dismiss-error
→ learned_patterns 테이블에 저장:

```

```
{ field, error_message, correct_interpretation, context }
```

→ 다음 검증 시:
 validator.py가 패턴 로드 → 매칭되면 해당 오류 억제
 → "[학습 패턴 적용됨]" info 경고로 투명 표시

4-6. Graceful Degradation

외부 서비스	있을 때	없을 때
OpenAI API	AI 매핑 + L3 검증	규칙 기반 매핑 + L1/L2만
Telegram	실시간 알림	알림 비활성, 검증 정상
Redis	세션 캐시	인메모리 세션

→ 핵심 기능(검증)은 외부 API 없이도 100% 동작

4-7. v3 → v4 개선사항

기능	v3	v4.1
인증	API Token	JWT + RBAC
개인정보	없음	PII 탐지/마스킹
감사 로깅	없음	전체 추적
워크플로우	없음	n8n/Telegram
DB	JSON 파일	SQLite
UI	기본	SheetEditor + 대시보드
학습	수동	자동

PART 5. 외워야 할 숫자

항목	숫자
기존 검증 시간	2~3시간
OneCheck 검증 시간	5분 (시스템 자체 5~10초)
매크로 에러체크 룰	21개 (100% 커버)
진단 질문	74개 (7섹션)
검증 단계	3단계 (L1+L2+L3)
L1 속도	0.1초
L2 속도	0.5초
L3 속도	3~5초
학습 효과	97건 → 4건 (93건 억제)
금액 허용 오차	±5%
최저임금 기준	2,060,740원
필수 필드	8개
개발 기간	2026.01~02
버전	4.1.0

PART 6. 시간 배분

파트	슬라이드	시간	핵심 메시지
문제+해결	1~3	2분	"3~4시간 → 5분"
UI 데모	4~7	4분	스크린샷, 편의성
로직+차별화	8~10	3분	3단계 검증+학습+보안
마무리	11~12	1분	비전+Q&A

PART 7. 데모 시나리오 (라이브 할 경우)

1. 로그인 (Complete 모드 선택)
2. 진단 질문 2~3 페이지 빠르게 넘기기
3. 파일 업로드 → AI 매핑 결과 보여주기
4. 3 색 검증 결과 보여주기
5. 오류 하나 "값 수정" → 스프레드시트 편집 → 적용 → 재검증
6. 오류 하나 "값 유지" → "학습됐습니다" 확인
7. (선택) 대시보드 + 텔레그램 알림

PART 8. 발표 중 강조 포인트

- 8. ★ "기존 매크로 21 개 100% 커버" — 업무 연속성
- 9. ★ "3 단계 검증" — 확실한 건 코드, 복잡한 건 AI
- 10. ★ "쓸수록 학습" — 97 건→4 건 실제 결과
- 11. ★ "개인정보 불필요 설계" — 보안
- 12. ★ "3~4 시간 → 5 분" — 효율
- 13. ★ "AI 없어도 100% 동작" — Graceful Degradation

OneCheck — Zero Touch Retirement Validation Platform

편리성 / 자동화 / 정확성